

C++ | CPL

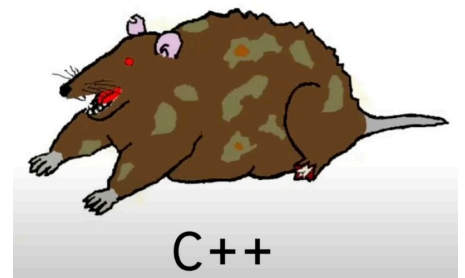
Summary

CONTENTS

1. Why C++?	2
2. Intro	2
2.1. Compilation	3
2.2. Declarations and definitions	3
2.2.1. #include guards	3
2.2.2. Functions	3
2.2.3. Values and references	3
2.3. Libraries	4
3. Values and Streams	4
3.1. Variables	4
3.2. Literal Values	4
3.3. Expressions	5
3.3.1. Type Conversion	5
3.3.2. Assignment Operation	5
3.3.3. Logic Operations	5
3.3.4. Floating Point Numbers (IEEE754)	5
3.3.5. Strings	6
3.4. Basic Streams	6
3.4.1. Errors	6
3.4.2. std::string	6
3.4.3. int	6
3.4.4. Boolean Conversion	7
3.4.5. IO Headers	7
4. Sequences and Iterators	8
4.1. std::array<T, N>	8
4.2. std::vector<T>	8
4.3. Iteration	9
4.3.1. Algorithms	9
Bibliography	12

1. WHY C++?

- Multi-paradigm language, now you have a thousand and one way to shoot yourself in your foot and obliterate your PC
- Being called stupid when you think that the move function actually moves things or if you confuse a const pointer with a pointer of const
- ISO standard: C++23. The higher the number the hotter the mess of poorly designed choices that have accumulated over decades into a chaotic pile of obsolescence
- Good excuse for alcoholism
- Zero-cost abstractions (if you like learning the whole stdlib by heart and don't mind the cost of debugging template errors for hours on end)
- Headaches (if you're a masochist)
-
- It's a never ending journey. Even if you dedicate your whole life to learning C++, you'll never reach the day where you'll know all of the stdlib.
- "Undefined behavior" is defined (in the standard)
- Ever wanted to have 100 different ways to do the same thing? Now you can
- Has a pretty mascot [\[1\]](#)



2. INTRO

- Implicitly returns 0
- Return types are written in front of the function name or as trailing return-types in declarations (because who needs consistency, amirite)

Nice and clean: `int main() { }` Insanity: `auto main() → int { }`

- Header files can be either `*.hpp` or `*.h`
- Regarding include order: It is good practice to include own-headers ("`"`") before system-headers (`<`). The first is usually used for the header files belonging to the project itself, while the latter is used for includes of the standard library and external libraries. However, ***this difference is only conventional, as any toolchain could just specify their own lookup rules for the two distinct kinds of includes.***

Do's	Don'ts
	Using global variables
	Redundant returns in <code>main()</code> that would be implicit
	<code>using namespace</code>
	Not initializing a variable upon definition using <code>{}</code>
	Not using <code>const</code> enough
	Using <code>sdt::cin</code> and <code>sdt::cout</code>
	Not checking <code>stream.good()</code>
	Not removing stream fail flag with <code>stream.clear()</code>
	Not using <code>#include</code> guards
	Using plain C-Arrays

<i>Do's</i>	<i>Don'ts</i>
	Using plain <code>for(int i; ...)</code> loops
	TODO: make this column real big for the lulz

2.1. COMPILATION

3 Phases of compilation:

Preprocessor: Textual replacement of preprocessor directives (`#include`)

Compiler: Translation of C++ code into machine code (source file to object file)

Linker: Combination of object files and libraries into libraries and executables

2.2. DECLARATIONS AND DEFINITIONS

All things with a name that you use in a C++ program must be declared before you can do so

- E.g. a function that you call (major difference from C)
- A type that you use for a variable (except some built-ins)
- A variable that you use

There can only be one definition of the same function (**One Definition Rule**/ODR), in contrast to declaration, which can be done several times.

2.2.1. `#include` guards

`#include` guards ensure that a header file is only included once. Necessary for functions and class type definitions, good practice either way.

```
#ifndef GUARD_HPP_
#define GUARD_HPP_
// ...
#endif /* GUARD_HPP_ */
```

2.2.2. Functions

Declarations

```
auto          <function-name>(<parameters>) → <return-type>;
<return-type> <function-name>(<parameters>);
```

Definitions

```
auto          <function-name>(<parameters>) → <return-type> { /*body*/}
<return-type> <function-name>(<parameters>)                  { /*body*/}
```

If a function has a non-void return type it must return a value on every path (or throw an exception).

2.2.3. Values and references

```
struct Point {
    int x;
    int y;
};
```

Copying vs. Sharing:

```
Point point{1, 20};
```

```
Point copiedPoint{point};
Point & sharedPoint{point};
```

```
auto copydPointParam(Point point) → Point { }
auto referencePointParam(Point & point) → Point { }
```

2.3. LIBRARIES

- [catch2](#)
- [boost](#)

TODO:
cmake

3. VALUES AND STREAMS

3.1. VARIABLES

```
<type> <variable-name>{<initial-value>;
```

Using = or {} for initialization with a value supplied we can have the compiler determine its type.

```
auto const i = 5;
```

Initialization might be omitted but that is bad practice and potentially dangerous.

```
double x;
```

Adding the const keyword in front of the name makes the variable a single-assignment variable, aka a constant which is immutable and must be initialized. Some constants are required to be fixed at compile time (**constexpr**). The keyword **const** also appears in other contexts.

```
int const theAnswer{42};
double constexpr pi{3.14};
```

You should use **const** whenever possible for non-member variables.

3.2. LITERAL VALUES

<i>Literal Example</i>	<i>Type</i>	<i>Value</i>
'a'	char	Letter a, value: 97
'\n'	char	<NL> character, value: 10
'\x0a'	char	<NL> character, value: 10
1	int	1
42L	long	42
5LL	long long	5
int{} // (not really a literal)	int	0 (default value)
1u	unsigned int	1

42uL	unsigned long	42
5uLL	unsigned long long	5
020	int	16 (octal 20)
0x1f	int	31 (hex 1F)
0XFULL	unsigned long long	15 (hex F)
0.f	float	0
.33	double	0.33
1e9	double	1000000000 (10 ⁹)
42.E-12L	long double	0.00000000042 (42*10 ⁻¹²)
.3L	long double	0.3
"hello"	char const [6]	Array of 6 chars: h e l l o <NUL>
"\012\n\\"	char const [4]	Array of 4 chars: <NL> <NL> <NUL>

3.3. EXPRESSIONS

	<i>Arithmetic</i>	<i>Bit-operators</i>	<i>Logic</i>
<i>Unary</i>	+ - ++ --	~	!
<i>Binary</i>	+ - * / %	& ^ << >>	&& < > ≤ ≥ = ≠
<i>Tertiary</i>			? :

- Fraction results of integer operations are always rounded down
- [Operator precedence](#)

3.3.1. Type Conversion

- Integer to boolean conversion 0 → false / every other value → true
- Automatic type conversion if values of different types are combined in an expression, **unless in braced initialization**
- Dividing integers by zero is **undefined behavior**

3.3.2. Assignment Operation

```
// vvvvv lvalue
myVar = 6 * 7;
// ^^^^^ rvalue
```

3.3.3. Logic Operations

Logical operators and conditional statements are generous to accept numeric values as statement of truth

```
// v returns boolean
if (a < b < c);
// ^ coalesces to 0 if false, 1 if true
```

3.3.4. Floating Point Numbers (IEEE754)

- Use double – usually most efficient on current hardware and default for floating point literals.
- Use float only if memory consumption is utmost priority and precision and range can be traded (on 64bit often not beneficial).

- Remember there are legal double values that are not numbers: NaN, +Inf, -Inf.
- Comparing floating points for equality (==) is usually wrong
 - In Catch2 you have the option to specify a matcher that checks in a relative range to the expected value `REQUIRE_THAT(actual, Catch::Matchers::WithinRel(expected, delta));`

3.3.5. Strings

`std::string` is C++'s type for representing sequences of char (which is often only 8 bit). For working with non-ASCII characters an external library is advised. It is mutable and iterable with iterators.

```
#include <string>
std::string statement{"Rust ftw"};
```

String literals like "ab" are not of type `std::string`, they're a null-terminated array of const characters (`char const[3]`). But "ab"s is an `std::string` but requires `using namespace std::literals`.

3.4. BASIC STREAMS

Streams aren't values, because they cannot be copied. So functions taking a stream object must take it as a reference.

Pre-defined globals: `std::cin` `std::cout`, should only be used in `main()`. "shift" operators read into variables or write values and can be chained:

```
std::cin >> x;
std::cout << "the value is " << x << '\n';
```

Output can be formatted using [I/O manipulators](#).

TODO:

W2 slides 43

3.4.1. Errors

Streams have a state that denotes if I/O was successful or not. If a previous read already failed, subsequent reads fail as well.

State Bit Set	Query	Entered
<none>	<code>is.good()</code>	initial <code>is.clear()</code>
failbit	<code>is.fail()</code>	formatted input failed
eofbit	<code>is.eof()</code>	trying to read at end of input
badbit	<code>is.bad()</code>	unrecoverable I/O error

- Formatted input on stream is must be checked for `is.fail()` and `is.bad()`
- If failed, `is.clear()` the stream and consume invalid input characters before continuing

3.4.2. std::string

Reading a `std::string` can not go wrong, unless the stream is already `!good()`

- The content of the `std::string` is replaced
- Maybe the `std::string` is empty after reading

3.4.3. int

Reading an `int` results in:

- No error recovery

- One wrong input puts the stream into status fail
- Characters remain in input

Robust way of reading an `int`:

```
auto readInt(std::istream & in) → int {
    std::string line{};
    while (getline(in, line)) {
        std::istringstream is{line};
        int res{-1};
        if (is >> res) {
            return res;
        }
    }
    return -1;
}
```

TODO:

W2 slides 41

3.4.4. Boolean Conversion

Result of `is >> res` is the `std::istream` object itself. The stream object converts to `bool` (in if and loop conditions):

- `true` if the last reading operation has been successful
- `false` if the last reading operation failed somehow (formatting, stream end or another problem)

`good()` for pre-checking “can I still read?” and `bool` conversion for post-checking “did I read correctly?”.

ios_base::iostate flags			basic_ios accessors					
eofbit	failbit	badbit	<code>good()</code>	<code>fail()</code>	<code>bad()</code>	<code>eof()</code>	operator <code>bool</code>	operator <code>!</code>
false	false	false	true	false	false	false	true	false
false	false	true	false	true	true	false	false	true
false	true	false	false	true	false	false	false	true
false	true	true	false	true	true	false	false	true
true	false	false	false	false	false	true	true	false
true	false	true	false	true	true	true	false	true
true	true	false	false	true	false	true	false	true
true	true	true	false	true	true	true	false	true

3.4.5. IO Headers

`iosfwd`: contains only the declarations for `std::ostream` and `std::istream`.

In header files (`.hpp`) this is usually sufficient when the streams are only used in function declarations

`istream` and `ostream`: contain the implementation of the corresponding stream and operators.

Usually, these are required in source files (`.cpp`) when the streams are actually used in functions

iostream: contains all of the above and additionally `std::cout`, `std::cin`, `std::cerr`.

This is only required in the source file containing the `main()` function, because only there the global standard IO variables shall be used

4. SEQUENCES AND ITERATORS

4.1. `STD::ARRAY<T, N>`

```
#include <array>
```

```
std::array<int, 6> emptyArray{};
```

Is a fixed-size Container

- T is a template type parameter (= placeholder for type)
- N is a positive integer, template non-type parameter (= placeholder for a value)

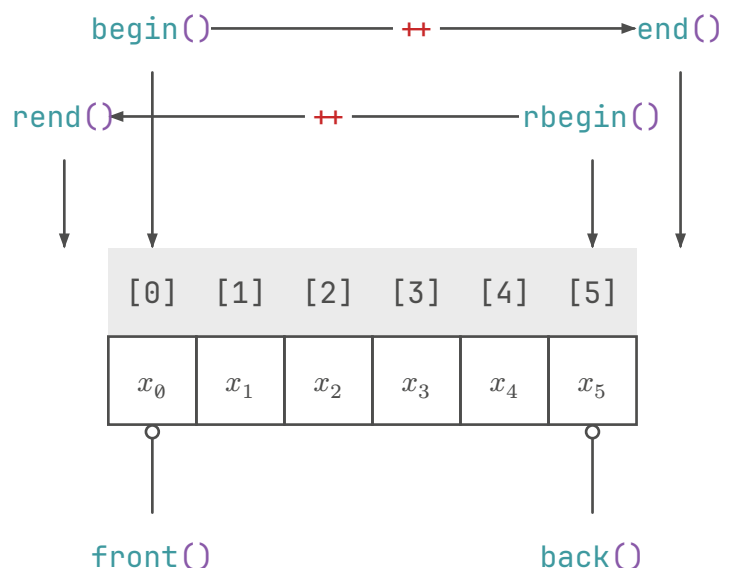
It can be initialized with a list of elements

- The size of an array must be known at compile-time and cannot be changed
- Otherwise, it contains N default-constructed elements

The size is bound to the array object and can be queried using `.size()`

Element access using subscript operator `[]` or `.at()`

- `.at()` throws `std::out_of_range` exception on invalid index access
- `[]` has undefined behavior on invalid index access



4.2. `STD::VECTOR<T>`

```
#include <vector>
```

```
std::vector<int> numbers{1, 2, 3, 4, 5};
```

Is a Container = contains its elements of type T (no need to allocate them). Works similar to an `java.util.ArrayList<T>` in java.

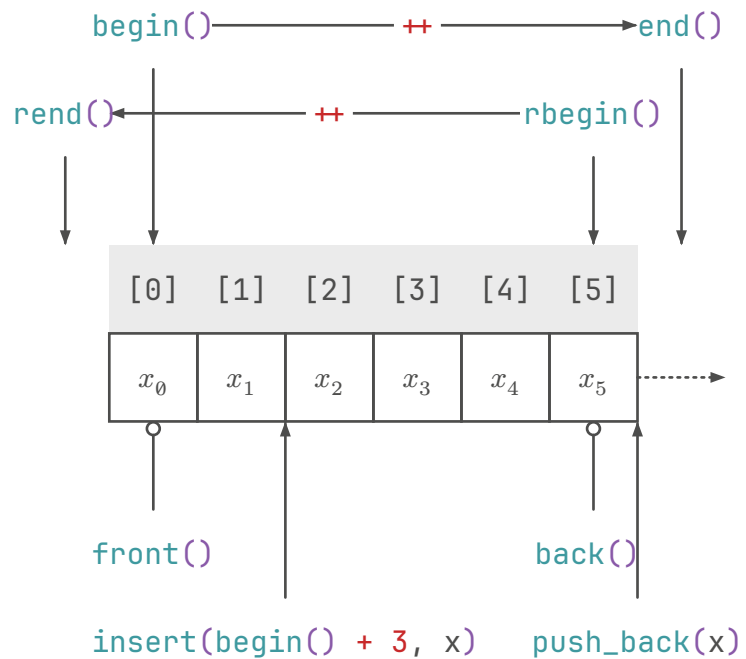
`std::vector` can be initialized with a list of elements

- The list can be empty:
`std::vector<double> vd{};`
- Other construction means might need parentheses (legacy)

When an initializer is given, the element type can be deduced! `std::vector{1, 2, 3, 4, 5};`

Parenthesis at definition allow providing initial size, when type of elements is not numeric `std::vector<std::string> words{6}`

Index variable type is
"unsigned" `std::size_t` or
`std::vector<T>::size_type`



TODO:

W3 S12

4.3. ITERATION

	<i>const</i>	<i>non-const</i>
<i>reference</i>	<code>for (auto const & cref : v) { }</code>	<code>for (auto & ref : v) { }</code>
<i>copy</i>	<code>for (auto const ccopy : v) { }</code>	<code>for (auto copy : v) { }</code>

Only use this if the iterator is required in the loop:

```
for (auto it = std::begin(v); it != std::end(v); ++it) {
    std::cout << (*it)++ << ", ";
}
```

Guarantee to just have read-only access: `std::cbegin()` and `std::cend()`

4.3.1. Algorithms

`#include <algorithm>`

Each algorithm takes iterator arguments: The range(s) of elements to apply an algorithm to is specified by iterators.

```
auto count_blanks(std::string s) → size_t {
    return std::count(s.cbegin(), s.cend(), ' ');
}
```

TODO:

W3 S20..

```
#include <numeric>
std::vector<int> v{5, 4, 3, 2, 1};
std::cout << std::accumulate(std::cbegin(v), std::cend(v), 0) << " = sum\n";

#include <iterator>
void printDistanceAndLength(std::string s) {
    std::cout << "distance: " << std::distance(s.begin(), s.end()) << '\n';
    std::cout << "in a string of length: " << s.size() << '\n';
}

auto print(int x) → void {
    std::cout << "print: " << x << '\n';
}

auto printAll(std::vector<int> v) → void {
    std::for_each(std::cbegin(v), std::cend(v), print);
}
```

What can we do if we want to print to a given `std::ostream`?

```
auto printAll(std::vector<int> v, std::ostream & out) → void {
    std::for_each(std::cbegin(v), std::cend(v), [&out](auto x) {
        out << "print: " << x << '\n';
    });
}
```

Lambdas:

```
[<capture>](<parameters>) → <return-type> {
    <statements>
}
```

Capture names variables taken from the surrounding scope, or define new ones (= copy, & -> reference, rename possible, type deduced)

Ranges:

```
auto printAll(std::vector<int> v, std::ostream & out) → void {
    std::ranges::for_each(v, [&out](auto x) {
        out << "print: " << x << '\n';
    });
}
```

Container → Iterators → Algorithms

When using the `std::copy` algorithm the target has to be an iterator too

```
std::copy(<input-begin-iterator>, <input-end-iterator>, <output-begin-iterator>);
std::ranges::copy(<input-range>, <output-begin-iterator>);
```

back_inserter S26+

```
std::vector<int> v{};
v.resize(10);
std::fill(std::begin(v), std::end(v), 2);
std::ranges::fill(v, 2)
```

```
// or
std::vector<int> v(10);
std::fill(std::begin(v), std::end(v), 2);
std::ranges::fill(v, 2)
// or
std::vector<int> v(10, 2);
```

BIBLIOGRAPHY

- [1] "C++ Mascot." Accessed: Sep. 12, 2026. [Online]. Available: https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink_800/feedshare-shrink_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC_GlaJFqlUgO03e-9B59Zpzu4-LYG90IM8k4UFE